ΑΣΦΑΛΕΙΑ ΣΥΣΤΗΜΑΤΩΝ–
Εργασία εξαμήνου

Εργασία: Ασφάλεια πιστοποίησης με συνθηματικά

Ιωάννης Μποζιονέλος

ΑΜ: 2022202300157

Κωνσταντίνος Βελέντζας

ΑΜ: 2022202300243

Αριθμός Ομάδας Χρηστών: 93

Τμήμα Πληροφορικής και Τηλεπικοινωνιών του Πανεπιστημίου Πελοποννήσου

8 Φεβρουαρίου 2026

Πίνακας περιεχομένων

Ερώτηση 1η – John the Ripper (JtR), τρόπος λειτουργίας και συγκριτική ανάλυση 3

1. Τι είναι το John the Ripper 3

2. Τρόπος λειτουργίας του John the Ripper 3

3. Συγκριτική ανάλυση με άλλα εργαλεία 3

Ερώτηση 2 4

Χώρος Αναζήτησης 4

Εκτίμηση χρόνου εκτέλεσης 5

Υπολογισμοί χρόνου εκτέλεσης 5

Screenshots 7

Ερώτηση 1η – John the Ripper (JtR), τρόπος λειτουργίας και συγκριτική ανάλυση

1. Τι είναι το John the Ripper

Το **John the Ripper (JtR)** είναι ένα εργαλείο ασφάλειας πληροφοριακών συστημάτων ανοικτού κώδικα που χρησιμοποιείται κυρίως για την **ανάκτηση και δοκιμή ισχύος κωδικών πρόσβασης** μέσω τεχνικών cracking. Χρησιμοποιείται ευρέως σε διαδικασίες ελέγχου ασφάλειας (penetration testing) και ανάλυσης ευπαθειών για τον εντοπισμό αδύναμων κωδικών σε ένα σύστημα.

2. Τρόπος λειτουργίας του John the Ripper

Η λειτουργία του βασίζεται στην προσπάθεια εύρεσης του αρχικού κωδικού μέσω σύγκρισης με αποθηκευμένα **hashes**. Γενικά η διαδικασία είναι η εξής:

1. **Εισαγωγή hashes**: Ο χρήστης δίνει στο πρόγραμμα hashes κωδικών από αρχεία συστήματος ή βάσεις δεδομένων.
2. **Δημιουργία υποψήφιων κωδικών**: Το εργαλείο δημιουργεί πιθανούς κωδικούς με διάφορες τεχνικές.
3. **Υπολογισμός hash**: Για κάθε πιθανό κωδικό υπολογίζεται το αντίστοιχο hash.
4. **Σύγκριση**: Αν το νέο hash ταιριάζει με το αρχικό, τότε έχει βρεθεί ο κωδικός.

Οι βασικές μέθοδοι επίθεσης που χρησιμοποιεί είναι:

- **Brute force** (δοκιμή όλων των πιθανών συνδυασμών)
- **Dictionary attack** (χρήση λιστών συχνών λέξεων/κωδικών)
- **Rule-based attacks** (παραλλαγές λέξεων με κανόνες)

Η διαδικασία γίνεται συνήθως offline, δηλαδή χωρίς σύνδεση στο σύστημα στόχο, κάτι που επιτρέπει μεγάλη ταχύτητα δοκιμών.

3. Συγκριτική ανάλυση με άλλα εργαλεία

α) Hashcat

Το **Hashcat** είναι ένα από τα πιο ισχυρά εργαλεία cracking και συχνά συγκρίνεται με το JtR.

Ομοιότητες

- Υποστηρίζει πολλές μορφές hash.
- Χρησιμοποιεί brute force και dictionary attacks.

- Χρησιμοποιείται σε penetration testing.

Διαφορές

- Το Hashcat εκμεταλλεύεται ιδιαίτερα τις **GPU** για πολύ μεγαλύτερη ταχύτητα.
- Έχει μεγάλη υποστήριξη multi-core και προηγμένους κανόνες cracking.
- Θεωρείται λιγότερο φιλικό για αρχάριους χρήστες.

Συμπέρασμα: Το Hashcat είναι πιο γρήγορο σε μεγάλες επιθέσεις, ενώ το JtR είναι πιο ευέλικτο και απλό για γενική χρήση.

β) THC Hydra

Το **THC Hydra** είναι εργαλείο brute force αλλά με διαφορετική φιλοσοφία.

Ομοιότητες

- Χρησιμοποιείται για έλεγχο αντοχής κωδικών.
- Υποστηρίζει brute force επιθέσεις.

Διαφορές

- Το Hydra κάνει **online επιθέσεις** σε υπηρεσίες (SSH, FTP, HTTP login κ.λπ.).
- Το JtR δουλεύει κυρίως με offline cracking hashes.
- Το Hydra ελέγχει συστήματα σε πραγματικό χρόνο, ενώ το JtR αναλύει αποθηκευμένα δεδομένα.

Συμπέρασμα: Το Hydra είναι κατάλληλο για testing login υπηρεσιών, ενώ το JtR για ανάλυση αποθηκευμένων κωδικών.

Ερώτηση 2η

Οι ομάδες εργασίας μας είναι η 93 οπότε κατεβάζουμε το αρχείο passwords_3.txt

Δεν χρησιμοποιήθηκε λεξικό καθώς οι κωδικοί είχαν συγκεκριμένη δομή και μικρό μήκος, χρησιμοποιήθηκε mask attack

Χώρος Αναζήτησης

1ος κωδικός – μόνο ψηφία

10 πιθανοί χαρακτήρες

Θέσεις: 4

Πλήθος: $10^5=100000$

2ος κωδικός – μόνο γράμματα

26 γράμματα

Πλήθος: 26^5=11.881.376

3ος κωδικός – μόνο γράμματα

Ίδιος υπολογισμός:

Πλήθος: 26^5=11.881.376

4ος κωδικός – πρώτο γράμμα a

Μένουν 4 γράμματα ελεύθερα:

Πλήθος: 26^4=456.976

5ος κωδικός – 2 ψηφία + 3 γράμματα

10^2*26^3

Πλήθος: 100*17.576=1.757.600

Εκτίμηση χρόνου εκτέλεσης

Ταχύτητα συστήματος:

1.510.000 hashes/sec

Τύπος:

$$\text{Χρόνος} = \frac{\text{Χώρος}}{\text{Ταχύτητα}}$$

Υπολογισμοί χρόνου εκτέλεσης

1ος κωδικός

100000/1510000=0.066sec

2ος κωδικός

11881376/1510000=7.87sec

3ος κωδικός

7.87sec

4ος κωδικός

456976/1510000=0.30sec

5ος κωδικός

1747600/1510000=1.16sec

Η θεωρητική εκτίμηση χρόνου βασίζεται στη μέγιστη ταχύτητα του εργαλείου (john --test) και υποθέτει ιδανικές συνθήκες εκτέλεσης. Στην πράξη, ο πέμπτος κωδικός χρειάστηκε 1 ώρα, 12 λεπτά και 55 δευτερόλεπτα για να βρεθεί, χρόνος σημαντικά μεγαλύτερος από τον θεωρητικό. Η διαφορά οφείλεται στη χρήση salted hashes, στην ταυτόχρονη επεξεργασία πολλών κωδικών, στη λειτουργία incremental attack και στους περιορισμούς του πραγματικού συστήματος (CPU scheduling, λειτουργικό σύστημα και background διεργασίες)

Screenshots

```
Command Prompt

Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

C:\Users\bozio_b46w8z5>cd C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run

C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>dir
 Volume in drive C is Windows
 Volume Serial Number is 8227-3CC9

 Directory of C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run

07/02/2026  09:17 μμ    <DIR>          .
07/02/2026  09:07 μμ    <DIR>          ..
07/02/2026  08:54 μμ            11.530 1password2john.py
07/02/2026  08:54 μμ            83.936 7z2john.pl
07/02/2026  08:54 μμ             3.897 adxcsouf2john.py
07/02/2026  08:54 μμ             2.403 aem2john.py
07/02/2026  08:54 μμ               894 aix2john.pl
07/02/2026  08:54 μμ             2.085 aix2john.py
07/02/2026  08:54 μμ         4.086.722 alnum.chr
07/02/2026  08:54 μμ         4.174.257 alnumspace.chr
07/02/2026  08:54 μμ         1.950.539 alpha.chr
07/02/2026  08:54 μμ             1.369 andotp2john.py
07/02/2026  08:54 μμ             3.456 androidbackup2john.py
07/02/2026  08:54 μμ             7.449 androidfde2john.py
07/02/2026  08:54 μμ             1.710 ansible2john.py
07/02/2026  08:54 μμ               732 apex2john.py
07/02/2026  08:54 μμ             2.028 applenotes2john.py
07/02/2026  08:54 μμ             1.448 aruba2john.py
07/02/2026  08:54 μμ         5.720.262 ascii.chr
07/02/2026  08:54 μμ             6.294 axcrypt2john.py
07/02/2026  08:54 μμ             9.216 base64conv.exe
07/02/2026  08:54 μμ            10.535 benchmark-unify
07/02/2026  08:54 μμ            10.174 bestcrypt2john.py
07/02/2026  08:54 μμ            33.770 bitcoin2john.py
07/02/2026  08:54 μμ            46.592 bitlocker2john.exe
07/02/2026  08:54 μμ             2.720 bitshares2john.py
07/02/2026  08:54 μμ             4.318 bitwarden2john.py
07/02/2026  08:54 μμ             7.755 bks2john.py
07/02/2026  08:54 μμ             2.686 blockchain2john.py
07/02/2026  08:54 μμ            12.288 calc_stat.exe
07/02/2026  08:54 μμ            26.741 ccache2john.py
07/02/2026  08:54 μμ             6.491 cisco2john.pl
07/02/2026  08:54 μμ               898 codepage.pl
07/02/2026  08:54 μμ            90.112 cprepair.exe
07/02/2026  08:54 μμ               881 cracf2john.py
07/02/2026  08:54 μμ            65.043 cygbz2-1.dll
07/02/2026  08:54 μμ            39.955 cygcrypt-0.dll
07/02/2026  08:54 μμ           198.163 cygcrypt-2.dll
07/02/2026  08:54 μμ         2.410.515 cygcrypto-1.0.0.dll
07/02/2026  08:54 μμ         2.498.067 cygcrypto-1.1.dll
```

```
C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john.exe
John the Ripper 1.9.0-jumbo-1 OMP [cygwin 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2019 by Solar Designer and others
Homepage: http://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]
--single[=SECTION[,..]]    "single crack" mode, using default or named rules
--single=:rule[,..]        same, using "immediate" rule(s)
--wordlist[=FILE] --stdin  wordlist mode, read words from FILE or stdin
                  --pipe   like --stdin, but bulk reads, and allows rules
--loopback[=FILE]          like --wordlist, but extract words from a .pot file
--dupe-suppression         suppress all dupes in wordlist (and force preload)
--prince[=FILE]            PRINCE mode, read words from FILE
--encoding=NAME            input encoding (eg. UTF-8, ISO-8859-1). See also
                           doc/ENCODINGS and --list=hidden-options.
--rules[=SECTION[,..]]     enable word mangling rules (for wordlist or PRINCE
                           modes), using default or named rules
--rules=:rule[;..]]        same, using "immediate" rule(s)
--rules-stack=SECTION[,..] stacked rules, applied after regular rules or to
                           modes that otherwise don't support rules
--rules-stack=:rule[;..]   same, using "immediate" rule(s)
--incremental[=MODE]       "incremental" mode [using section MODE]
--mask[=MASK]              mask mode using MASK (or default from john.conf)
--markov[=OPTIONS]         "Markov" mode (see doc/MARKOV)
--external=MODE            external mode or word filter
--subsets[=CHARSET]        "subsets" mode (see doc/SUBSETS)
--stdout[=LENGTH]          just output candidate passwords [cut at LENGTH]
--restore[=NAME]           restore an interrupted session [called NAME]
--session=NAME             give a new session the NAME
--status[=NAME]            print status of a session [called NAME]
--make-charset=FILE        make a charset file. It will be overwritten
--show[=left]              show cracked passwords [if =left, then uncracked]
--test[=TIME]              run tests and benchmarks for TIME seconds each
--users=[-]LOGIN|UID[,..]  [do not] load this (these) user(s) only
--groups=[-]GID[,..]       load users [not] of this (these) group(s) only
--shells=[-]SHELL[,..]     load users with[out] this (these) shell(s) only
--salts=[-]COUNT[:MAX]     load salts with[out] COUNT [to MAX] hashes
--costs=[-]C[:M][,...]     load salts with[out] cost value Cn [to Mn]. For
                           tunable cost parameters, see doc/OPTIONS
--save-memory=LEVEL        enable memory saving, at LEVEL 1..3
--node=MIN[-MAX]/TOTAL     this node's number range out of TOTAL count
--fork=N                   fork N processes
--pot=NAME                 pot file to use
--list=WHAT                list capabilities, see --list=help or doc/OPTIONS
--devices=N[,..]           set OpenCL device(s) (see --list=opencl-devices)
--format=NAME              force hash of type NAME. The supported formats can
```

```
C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john.exe hash.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-opencl"
Use the "--format=md5crypt-opencl" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 5 password hashes with 5 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Remaining 4 password hashes with 4 different salts
Will run 8 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Warning: Only 94 candidates buffered for the current salt, minimum 192 needed for performance.
Warning: Only 93 candidates buffered for the current salt, minimum 192 needed for performance.
Warning: Only 94 candidates buffered for the current salt, minimum 192 needed for performance.
Proceeding with wordlist:password.lst, rules:Wordlist
alina            (user33)
duck             (user23)
joe              (user13)
Warning: Only 172 candidates left, minimum 192 needed for performance.
Proceeding with incremental:ASCII
3g 0:00:02:11  3/3 0.02289g/s 161210p/s 161271c/s 161271C/s djl0384..djlis26
3g 0:00:07:44  3/3 0.006457g/s 156121p/s 156138c/s 156138C/s htf084..htxucf
3g 0:00:12:17  3/3 0.004065g/s 154703p/s 154714c/s 154714C/s 2t009k..2t0rni
3g 0:00:31:32  3/3 0.001585g/s 150727p/s 150731c/s 150731C/s r1pel2c..r1pht1.
3g 0:00:53:53  3/3 0.000927g/s 147262p/s 147265c/s 147265C/s jjmero2u..jjmediv4
3g 0:01:07:30  3/3 0.000740g/s 144860p/s 144862c/s 144862C/s hlualls..hluaid1
a0r9e            (user43)
4g 0:01:12:55 DONE 3/3 (2026-02-07 23:18) 0.000914g/s 143962p/s 143964c/s 143964C/s a0r4*..a0l1e
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

```
C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john --test
Will run 8 OpenMP threads
Benchmarking: descrypt, traditional crypt(3) [DES 256/256 AVX2]... (8xOMP) /
DONE
Many salts:     13492K c/s real, 2121K c/s virtual
Only one salt:  11390K c/s real, 1828K c/s virtual

Benchmarking: bsdicrypt, BSDI crypt(3) ("_J9..", 725 iterations) [DES 256/256 AVX2]... (8xOMP) DONE
Speed for cost 1 (iteration count) of 725
Warning: "Many salts" test limited: 255/256                                          DONE
Warning: "Many salts" test limited: 127/256virtual
Many salts:lt:  145793 c/s real, 22061 c/s virtual
Only one salt:  152049 c/s real, 20854 c/s virtual
Benchmarking: md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3]... (8xOMP) |
Benchmarking: md5crypt-long, crypt(3) $1$ (and variants) [MD5 32/64]... (8xOMP) DONE
Raw:    24064 c/s real, 3398 c/s virtual

Benchmarking: bcrypt ("$2a$05", 32 iterations) [Blowfish 32/64 X3]... (8xOMP) DONE
Speed for cost 1 (iteration count) of 32
Raw:    5023 c/s real, 742 c/s virtual

Benchmarking: scrypt (16384, 8, 1) [Salsa20/8 128/128 AVX]... (8xOMP) DONE
Speed for cost 1 (N) of 16384, cost 2 (r) of 8, cost 3 (p) of 1
Raw:    142 c/s real, 19.8 c/s virtual

Benchmarking: LM [DES 256/256 AVX2]... (8xOMP) DONE
Raw:    50433K c/s real, 23081K c/s virtual

Benchmarking: AFS, Kerberos AFS [DES 48/64 4K]... DONE
Short:  354794 c/s real, 364670 c/s virtual
Long:   1318K c/s real, 1355K c/s virtual

Benchmarking: tripcode [DES 256/256 AVX2]... (8xOMP) DONE
Raw:    7731K c/s real, 1582K c/s virtual

Benchmarking: AndroidBackup [PBKDF2-SHA1 256/256 AVX2 8x AES]... (8xOMP) DONE
Speed for cost 1 (iteration count) of 10000
Raw:    1777 c/s real, 225 c/s virtual

Benchmarking: adxcrypt [IBM/Toshiba 4690 - ADXCRYPT 32/64]... (8xOMP) DONE
```

```
C:\john-1.9.0-jumbo-1-win64\john-1.9.0-jumbo-1-win64\run>john --show hash.txt
user03:3284
user13:joe
user23:duck
user33:alina
user43:a0r9e

5 password hashes cracked, 0 left
```

ΑΣΦΑΛΕΙΑ ΣΥΣΤΗΜΑΤΩΝ–
Εργασία εξαμήνου

Η παρούσα εργασία ολοκληρώθηκε με στόχο την πλήρη και τεκμηριωμένη ανάλυση του θέματος!

Ευχαριστώ θερμά για την προσοχή και το ενδιαφέρον σας!

Με εκτίμηση,

Ιωάννης Μποζιονέλος & Κωνσταντίνος Βελέντζας